

Code 1:

```
import hashlib
import random
import math
```

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True
```

```
def generate_prime():
    while True:
        n = random.randint(100, 300)
        if is_prime(n):
            return n
```

```
def generate_keys():
    p = generate_prime()
    q = generate_prime()

    while p == q:
        q = generate_prime()
```

```
n = p * q
phi = (p - 1) * (q - 1)
```

```
e = 65537
```

```
if math.gcd(e, phi) != 1:
    e = 3
    while math.gcd(e, phi) != 1:
        e += 2
```

```
d = pow(e, -1, phi)
```

```
return (e, n), (d, n)
```

```
def sign_message(message, private_key):
    d, n = private_key
```

```
    hash_value = hashlib.sha256(message.encode()).hexdigest()
```

```
    Hariprasad Vishwakarma
```

```
    T127
```

```
    TYCS
```

```

hash_number = int(hash_value, 16)

hash_number = hash_number % n

signature = pow(hash_number, d, n)

return signature

def verify_signature(message, signature, public_key):
    e, n = public_key

    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16)

    hash_number = hash_number % n

    recovered_hash = pow(signature, e, n)

    return recovered_hash == hash_number

print("==== RSA DIGITAL SIGNATURE =====")

public_key, private_key = generate_keys()

print("\nPublic Key:", public_key)
print("Private Key:", private_key)

message = input("\nEnter message: ")

signature = sign_message(message, private_key)

print("\nDigital Signature:", signature)

if verify_signature(message, signature, public_key):
    print("\nSignature Verification: SUCCESS")
    print("Message is authentic and unchanged.")
else:
    print("\nSignature Verification: FAILED")

modified_message = input("\nEnter message again to verify: ")

if verify_signature(modified_message, signature, public_key):
    print("Modified Message Verification: SUCCESS")

```

Hariprasad Vishwakarma  
T127  
TYCS

else:

```
print("Modified Message Verification: FAILED")
print("Message was changed.")
```

Output1:

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: D:/T127/cyber security4.py =====
===== RSA DIGITAL SIGNATURE =====

Public Key: (65537, 32437)
Private Key: (2177, 32437)

Enter message: hariprasad

Digital Signature: 22714

Signature Verification: SUCCESS
Message is authentic and unchanged.

Enter message again to verify: hariprasad
Modified Message Verification: SUCCESS

>>>
```

Code2:import hashlib

import random

import math

import tkinter as tk

from tkinter import messagebox

# ===== RSA FUNCTIONS =====

def is\_prime(n):

if n < 2:

return False

for i in range(2, int(math.sqrt(n)) + 1):

if n % i == 0:

return False

return True

def generate\_prime():

while True:

n = random.randint(1000, 5000)

if is\_prime(n):

return n

Hariprasad Vishwakarma

T127

TYCS

```

def generate_keys():
    p = generate_prime()
    q = generate_prime()

    while p == q:
        q = generate_prime()

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    if math.gcd(e, phi) != 1:
        e = 3
        while math.gcd(e, phi) != 1:
            e += 2

    d = pow(e, -1, phi)

    return (e, n), (d, n)

def sign_message(message, private_key):
    d, n = private_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
    signature = pow(hash_number, d, n)
    return signature

def verify_signature(message, signature, public_key):
    e, n = public_key
    hash_value = hashlib.sha256(message.encode()).hexdigest()
    hash_number = int(hash_value, 16) % n
    recovered_hash = pow(signature, e, n)
    return recovered_hash == hash_number

# ===== GUI =====
public_key, private_key = generate_keys()
signature = None

def sign():
    global signature
    msg = entry_msg.get()
    if msg == "":
        Hariprasad Vishwakarma
        T127
        TYCS

```

```

        messagebox.showwarning("Error", "Enter a message")
        return

    signature = sign_message(msg, private_key)
    text_output.delete(1.0, tk.END)
    text_output.insert(tk.END, f"Public Key: {public_key}\n")
    text_output.insert(tk.END, f"Private Key: {private_key}\n\n")
    text_output.insert(tk.END, f"Message: {msg}\n")
    text_output.insert(tk.END, f"Signature: {signature}\n")

def verify():
    msg = entry_verify.get()
    if signature is None:
        messagebox.showwarning("Error", "Sign a message first")
        return

    result = verify_signature(msg, signature, public_key)

    if result:
        messagebox.showinfo("Verification", "✅ SUCCESS\nMessage is authentic")
    else:
        messagebox.showerror("Verification", "❌ FAILED\nMessage modified")

root = tk.Tk()
root.title("RSA Digital Signature")
root.geometry("500x400")

tk.Label(root, text="Enter Message:").pack()
entry_msg = tk.Entry(root, width=50)
entry_msg.pack()

tk.Button(root, text="Sign Message", command=sign).pack(pady=10)

tk.Label(root, text="Verify Message:").pack()
entry_verify = tk.Entry(root, width=50)
entry_verify.pack()

tk.Button(root, text="Verify Signature", command=verify).pack(pady=10)

text_output = tk.Text(root, height=10, width=60)
text_output.pack(pady=10)

root.mainloop()

```

Output2:

